



INDIAN INSTITUTE OF TECHNOLOGY
KHARAGPUR

Signature of the Invigilator

Please fill up carefully the boxes provided below

EXAMINATION (End Semester)										SEMESTER (Spring)																																																											
Roll Number										Section										Name																																																	
Subject Number										Subject Name										Algorithms-I																																																	
C										S										2										1										2										0										3									
Name of the Department / Center of the Student																																																																					

Instructions and Guidelines to Students Appearing in the Examination

1. Ensure that you have occupied the seat as per the examination schedule.
2. Ensure that you do not have a mobile phone or a similar gadget with you even in switched off mode. Note that loose papers, notes, books should not be in your possession, even if those are irrelevant to the paper you are writing.
3. Data book, codes or any other materials are allowed only under the instruction of the paper-setter.
4. Use of instrument box, pencil box and non-programmable calculator is allowed during the examination. However, exchange of these items is not permitted.
5. Additional sheets for rough work or writing answers, graph papers and relevant tables will be provided on request. **Those need not be submitted for Algorithms-I (CS21203).**
6. **Carefully read and follow instructions on the question paper.** Write on both sides of the answer script and do not tear off any page. Report to the invigilator if the answer script has torn page(s).
7. Show the identity card whenever asked for by the invigilator. It is your responsibility to ensure that your attendance is recorded by the invigilator.
8. You may leave the examination hall for wash room or for drinking water, but not before one hour after the commencement of the examination. Record your absence from the examination hall in the register provided. Smoking and consumption of any kind of beverages is not allowed inside the examination hall.
9. After the completion of the examination, do not leave the seat until the invigilator collects the answer script.
10. During the examination, either inside the examination hall or outside the examination hall, gathering information from any kind of sources or any such attempts, exchange or helping in exchange of information with others or any such attempts will be treated as adopting 'unfair means.' Do not adopt 'unfair means' and do not indulge in unseemly behavior as well.

Violation of any of the instructions may lead to disciplinary action of varied nature.

To be filled in by the examiner										
Question	1	2	3	4	5	6	7	8	9	Total / 85
Marks obtained										
										Signature of Student
Total marks obtained (in words)			Signature of Scrutineer					Signature of Examiner		
Time: Tue, Apr 23, 2024, 2-4pm			Students: 68			Venue: NR-122/221/222 (G,I)				

1. Answer all the questions on the question paper itself in the spaces provided.
2. Credits will be given for neat and precise answering.
3. For **all questions**, you have to write only the answers. No explanation is needed. No part marks are assured; you get full marks only if your answer is fully correct.
4. Unless mentioned otherwise, *time complexity to solve a problem P* means the worst-case time complexity by the best-known algorithm for *P*.

1. Operations on a red-black tree T are indicated as follows: *a*) NuT: an empty red-black T is created, *b*) I k : key k is inserted in T , *c*) D k : key k is deleted from T (giving preference to using largest key from left subtree, if present) and *d*) PrT: preorder traversal of T is printed where each node is depicted as $\langle c, k \rangle$, where c is the colour of the node storing the key k .

Based on the given sequence of insertions and deletions, show the preorder traversal whenever PrT appears.

NuT, I 10, I 20, I 30, I 40, I 50, PrT,

3

I 32, I 34, I 36, PrT,

3

I 60, I 70, D 10, PrT,

3

D 60, D 70, D 20, PrT

3

2. Operations on a 2–3 tree T are indicated as follows: *a*) NuT: an empty 2–3 T is created, *b*) I k : key k is inserted in T , *c*) D k : key k is deleted from T (giving preference to using largest key from preceeding left subtree, if present) and *d*) PrT: preorder¹ traversal of T is printed.

Based on the given sequence of insertions and deletions, show the preorder traversal whenever PrT appears.

NuT, I 10, I 20, I 30, I 40, I 50, PrT,

3

I 32, I 34, I 36, PrT,

3

I 60, I 70, D 10, PrT,

3

D 60, D 70, D 20, PrT

3

¹The preorder traversal of a 2–3 tree rooted at T is defined as follows: *i*) ϵ if T is empty (NULL), otherwise *ii*) if the root node of T has one key k_1 and subtrees T_1 and T_2 then $\langle k_1 \rangle$ followed by the preorder traversals of T_1 and T_2 , *iii*) if the root node of T has two keys k_1 and k_2 and subtrees T_1 , T_2 and T_3 then $\langle k_1, k_2 \rangle$ followed by the preorder traversals of T_1 , T_2 and T_3 .

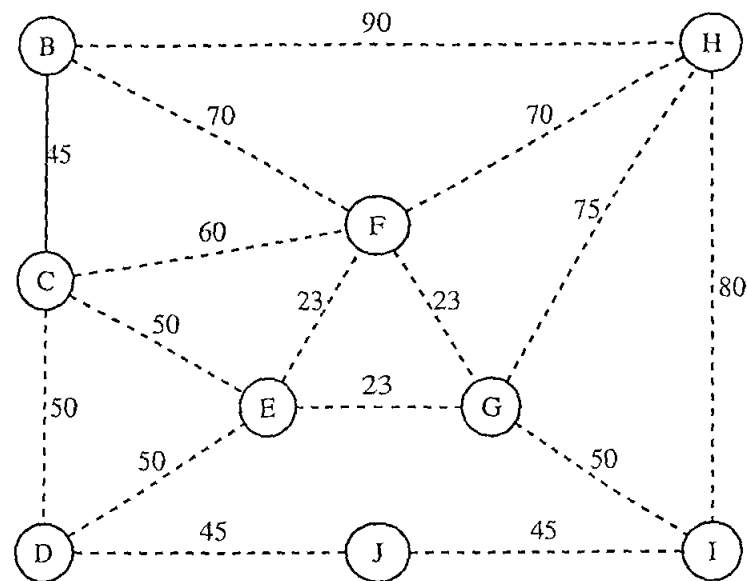


Figure 1: Graph for computing the maximum cost spanning tree

3. For the graph in figure 1, give the sequence of selecting edges that will yield the maximum cost spanning tree following the analogue of Kruskal's spanning tree construction principle. In case of ties, resolve ties in favour of the edge having the lowest node index. Highlight the spanning tree on the figure and call it T_1 .

8

Principle for choosing edges: _____

Edge-1: _____

Edge-2: _____

Edge-3: _____

Edge-4: _____

Edge-5: _____

Edge-6: _____

Edge-7: _____

Edge-8: _____

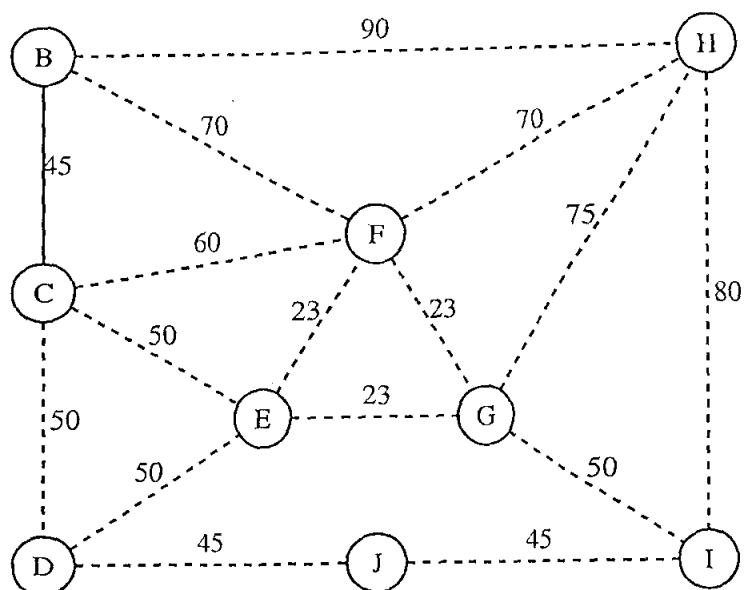


Figure 2: Graph for computing the maximum cost spanning tree

4. Starting at node D , for the graph in figure 2 (same as that in figure 1), give the sequence of selecting nodes and the associated edges that will yield the maximum cost spanning tree following the analogue of Prim's spanning tree construction principle. In each step, report the node cost and also the edge that is introduced as a result of selecting the node. Resolve ties in favour of the highest node index. Highlight the spanning tree on the figure and call it T_2 .

8

Principle for choosing nodes and edges: _____

Node-1:

Node Costs	B	C	D	E	F	G	H	I	J
------------	---	---	---	---	---	---	---	---	---

 $D, \langle \rangle, 0$

Node-2:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-3:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-4:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-5:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-6:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-7:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-8:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

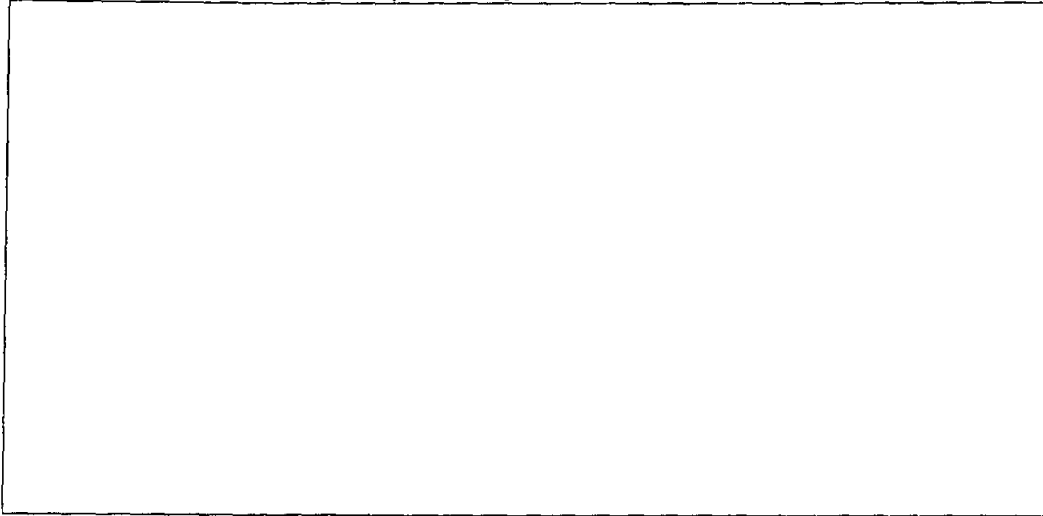
Node-9:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

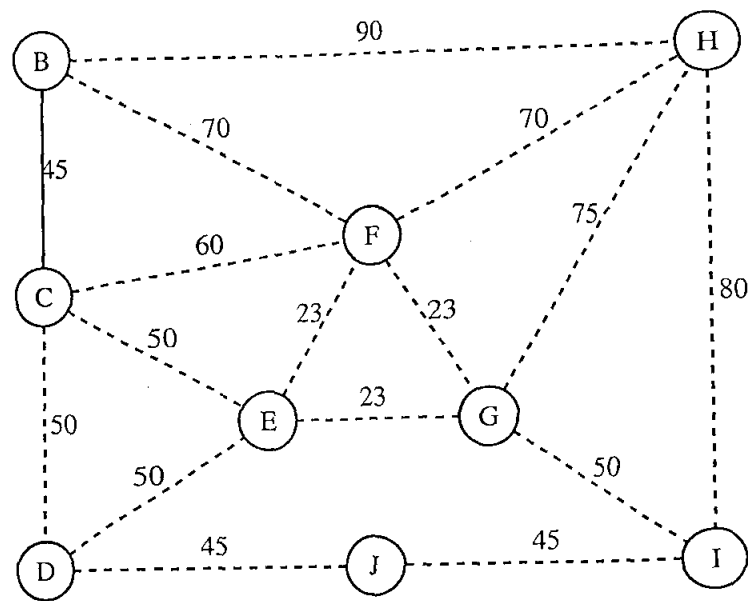
5. Trees T_1 and T_2 are not identical. Let's assume that T_2 obtained by the analogue of Prim's algorithm is indeed optimal, whereas the optimality of T_1 needs to be established. Giving justifications along the lines of the proof of Kruskal's algorithm, depict the first (non-trivial) step of the proof to establish that T_1 is also optimal, utilising T_2 .

5

- First, very briefly, state the principle of the proof.



- Now present the first (non-trivial) step of the proof with respect to T_1 and T_2 .

Figure 3: Graph for computing the shortest path tree rooted at D

6. Starting at node D , for the graph in figure 3, give the sequence of selecting nodes and the associated edges that will yield the shortest path tree. In each step, report the node cost and also the edge that is introduced as a result of selecting the node. In case of ties, select the edge having highest node index. Highlight the shortest path tree rooted at node D on the figure.

8

Principle for choosing nodes and edges: _____

Node-1:

Node Costs	B	C	D	E	F	G	H	I	J
------------	---	---	---	---	---	---	---	---	---

 $D, \langle \rangle, 0$

Node-2:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-3:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-4:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-5:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-6:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-7:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-8:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

Node-9:

Node Costs			0						
------------	--	--	---	--	--	--	--	--	--

7. Consider an idealised hashing and rehashing scheme where the initial hash table has a size of 1 and every insert has a cost of 1. If the table is full, a new table of double the size of the original table is used and all elements of the older table are mapped to the new table at a cost of 1 per table entry.

i) list the insertion costs of the elements 3, 1, 6, 7, 2, 8, 4 and 5 following the given scheme:

5

3: _____

1: _____

6: _____

7: _____

2: _____

8: _____

4: _____

5: _____

- ii) Considering the insertions done above, identify the invariant (say I) that is satisfied with respect to the maximum cost per element for inserting 2^k elements with rehashing. $1 \leq k \leq 3$.

5

$k = 1$: _____

$k = 2$: _____

$k = 3$: _____

- iii) Assuming that this invariant (I) holds for inserting 2^j elements, $1 \leq j \leq k$ with rehashing, show that it also holds for inserting 2^j elements, $1 \leq j \leq k + 1$ with rehashing.

5

8. i) For the search pattern aabaababb, compute the failure function in the table below.

Indexed search pattern								
1	2	3	4	5	6	7	8	9
a	a	b	a	a	b	a	b	b
Failure function								

7

- ii) Fill in the blanks to complete the pseudo code of the function for pattern matching using the KMP algorithm for finding the occurrence of a pattern given string `char P[1..M]` (a string `P` of `M` symbols, numbered from 1 to `M`) in a text string `char T[1..N]` (a string `T` of `N` symbols numbered from 1 to `N`) using the failure function (given via `int fail[1..M]`). NB: All the blank spaces may not have to be filled to complete the pseudo code.

```

int matchKMP (int M, char P[1..M], int fail[1..M],
              int N, char T[1..N]) {
    // NB: C syntax and indexing are not followed strictly
    // NB: T [], P[] and fail[] are indexed from 1, by convention
    // Return value: position of match or NoMatch for failure
    int j ← 1; // starting position in P

```

```

    for int i ← 1 to _____ do // scan through T

```

```

        while ( _____ )

```

```

            done

```

```

        endfor

```

```

    return NoMatch

```

```

}

```

5

9. You are given an undirected weighted graph representing roads between towns, annotated with the permissible vehicle width (the maximum width of a vehicle that can pass through the road). Fill in the blanks to compute the maximum safe widths of vehicles between all pair of towns. 5

```
void safeVehicleWidths (int n, int A[n][n]) {  
    // A is symmetric as the graph is undirected  
    // A[i][j]:  if road present between towns i and j,  
    // then the permissible vehicle width for that road,  
    // otherwise 0,  
    // furthermore:  A[i][i]=MAXINT, the maximum possible width
```

```
    int i, j, k;
```

```
    for (k=0; k<n; k++)
```

```
        for (i=0; i<n; i++)
```

```
            for (j=0; j<i; j++) {
```

```
                _____  
                _____  
                _____  
                _____  
                _____
```

```
            }
```

```
    }
```